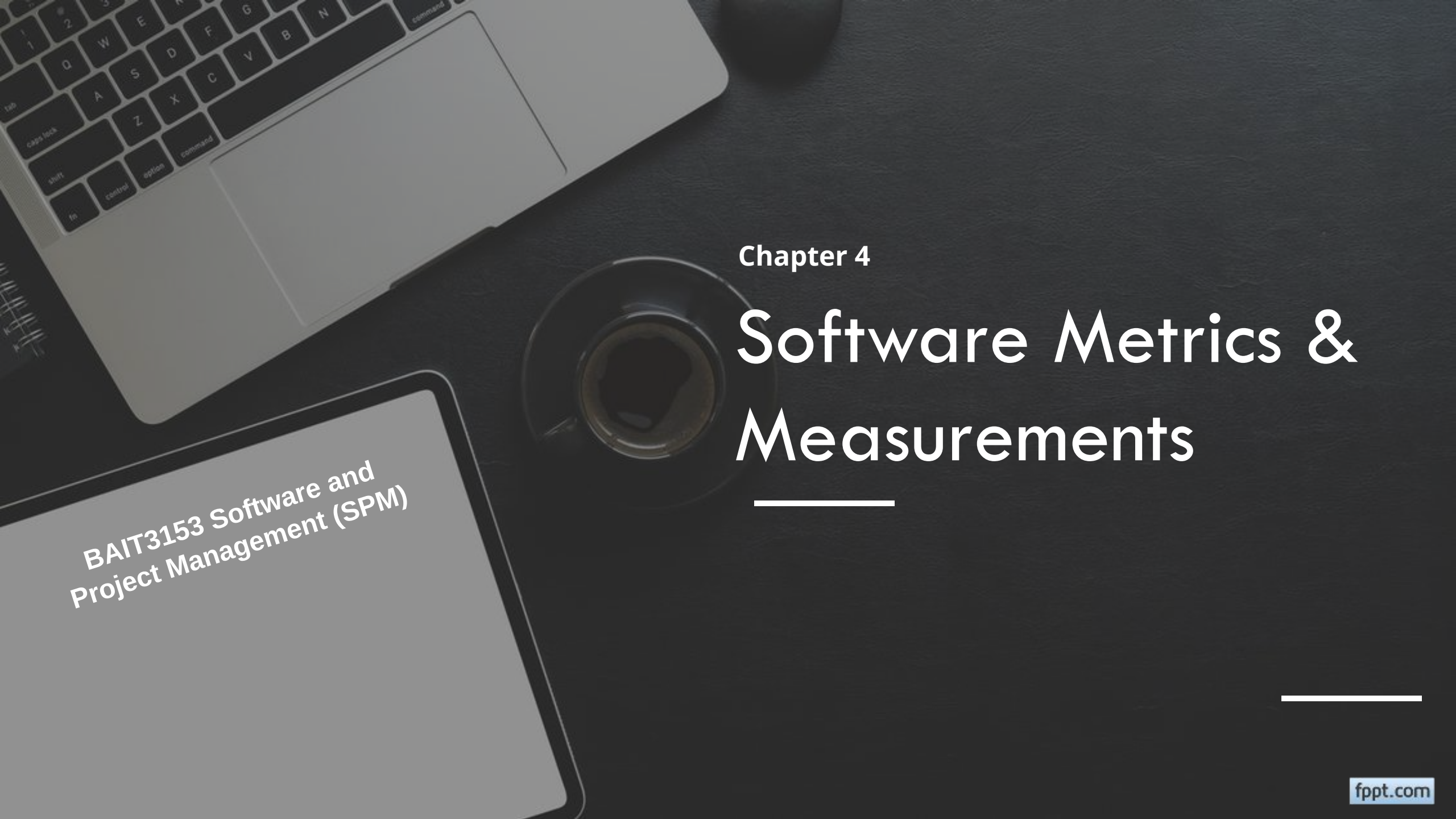


Chapter 4

Software Metrics & Measurements

BAIT3153 Software and Project Management (SPM)

Table of Contents

"is the **quantitative** measures of **'something'** "

e.g. "**Project A**"

- Total lines of code(LOC),
- Total pages of documentation,
- No of defects reported by customers 1 week after installation
- etc

The act of **finding out the amount or size of "something"**

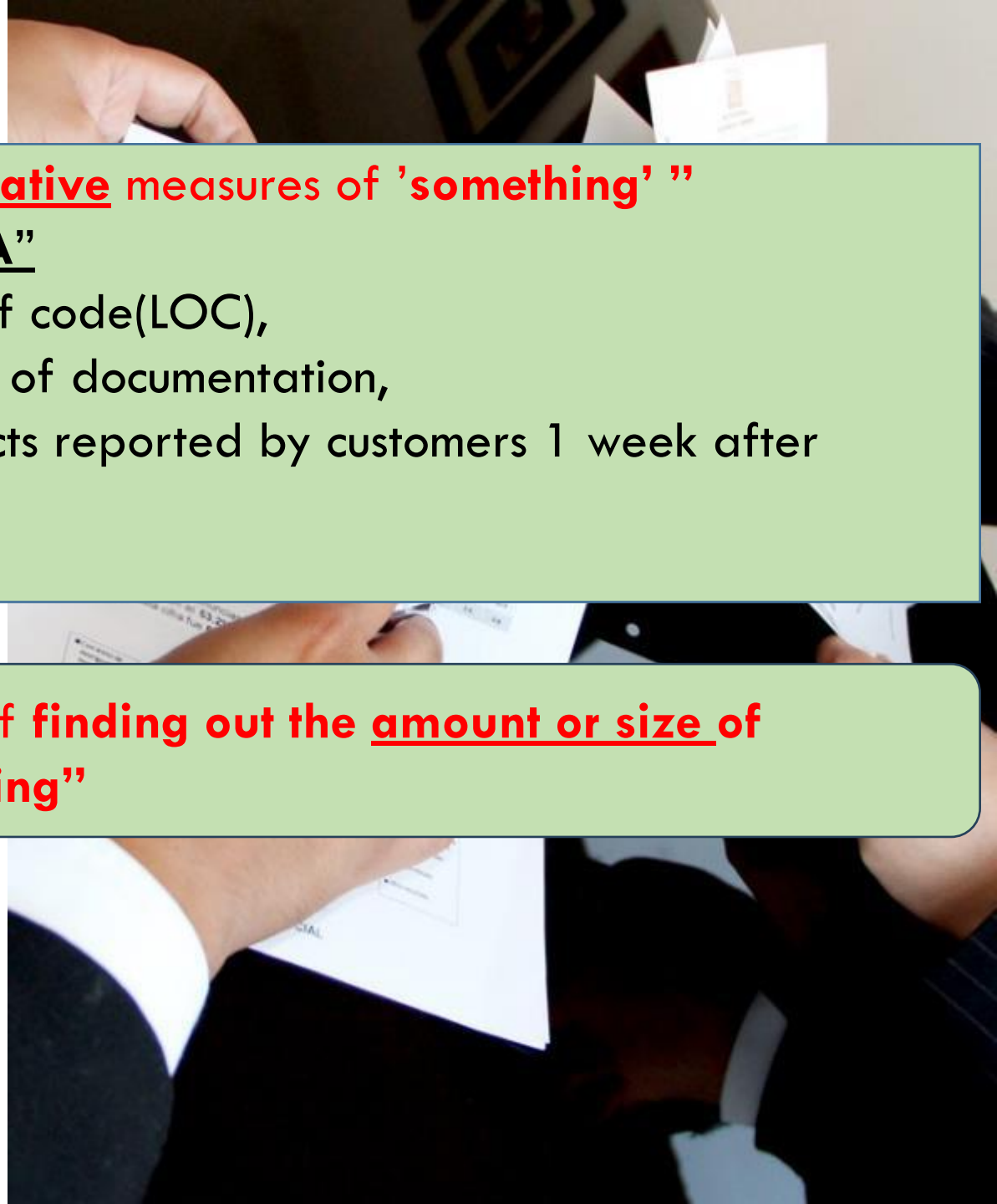


Table of Contents

"Is the **quantitative** measures of '**something**' "

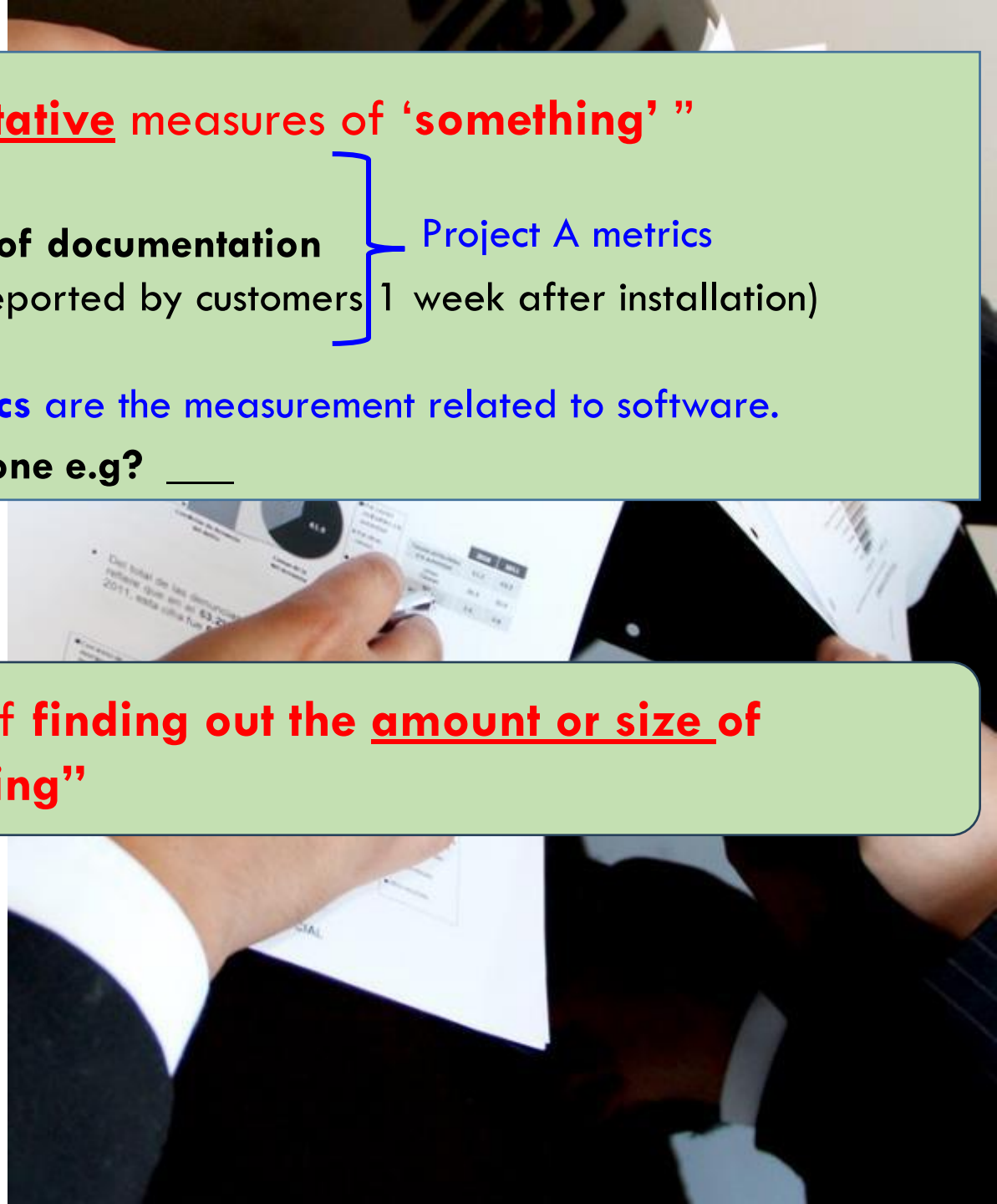
- **25,000 LOC**
- **2,000 Pages of documentation**
- **10 defects** (reported by customers 1 week after installation)

} Project A metrics

Software Metrics are the measurement related to software.

Can you give one e.g? ___

The act of **finding out the amount or size of "something"**



4.1 Introduction

- Software measurement is concerned with finding a numeric value for quantitative evaluation of an attribute of a **software <u>product</u> or <u>process</u>**.
- Applied in software process with the intent of **<u>improving</u>** it on a continuous basis.
- This allows for objective comparisons between products, techniques and processes.

4.2 Software Metrics

- **Examples of Software Metrics**
  (see next slide)
- Use of Software Metrics
- Indicators
- Types of Metrics

4.2 Software Metrics

- A quantitative measure of the degree to which a system, component or process possesses a given attribute.

- **Examples of software metrics** *(below are normalized metrics)*:
  - Errors/KLOC
  - Defects/KLOC
  - Cost/LOC
  - Pages of document/KLOC
  - Errors/person-month
  - LOC/person-month

	Non normalized metrics			
	Project A	Project B		
Total Line of Code (LOC)	306,000	256,000		
Errors	288	233		
Defects	30	20		
Pages of Documentation	205	180		

P(A) has 50% more defects than P(B)
*(30-20)/20 * 100%*

4.2 Software Metrics

- A quantitative measure of the degree to which a system, component or process possesses a given attribute.
- Examples of normalised metrics
  - **Errors/KLOC**
  - **Defects/KLOC**
  - Cost/**LOC**
  - **Pages of document/KLOC**
  - **Errors/person-month**
  - **LOC/person-month**

Means compare errors and defects and other "things" **between projects based on per 1000 LOC**

In every 1000 loc, there is 0.9412 error in P(A)

	Non normalized		Normalized with KLOC	
	Project A	Project B	Project A	Project B
Total Line of Code (LOC)	306,000	256,000		
Errors	288	233	288 / 306 = 0.9412	233 / 256 = 0.9102
Defects	30	20	30 / 306 = 0.0980	20 / 256 = 0.0781
Pages of Documentation	205	180	205/306 = 0.6699	180/256 = 0.7031

P(B): in every 1000 loc there is 0.9102 error

Non normalised metric shows that P(A) has **50%** more defects than P(B) *(30-20)/20 * 100*

In every 1000 LOC:
P(A) has 0.098 defects, P(B) has 0.0781 defects.

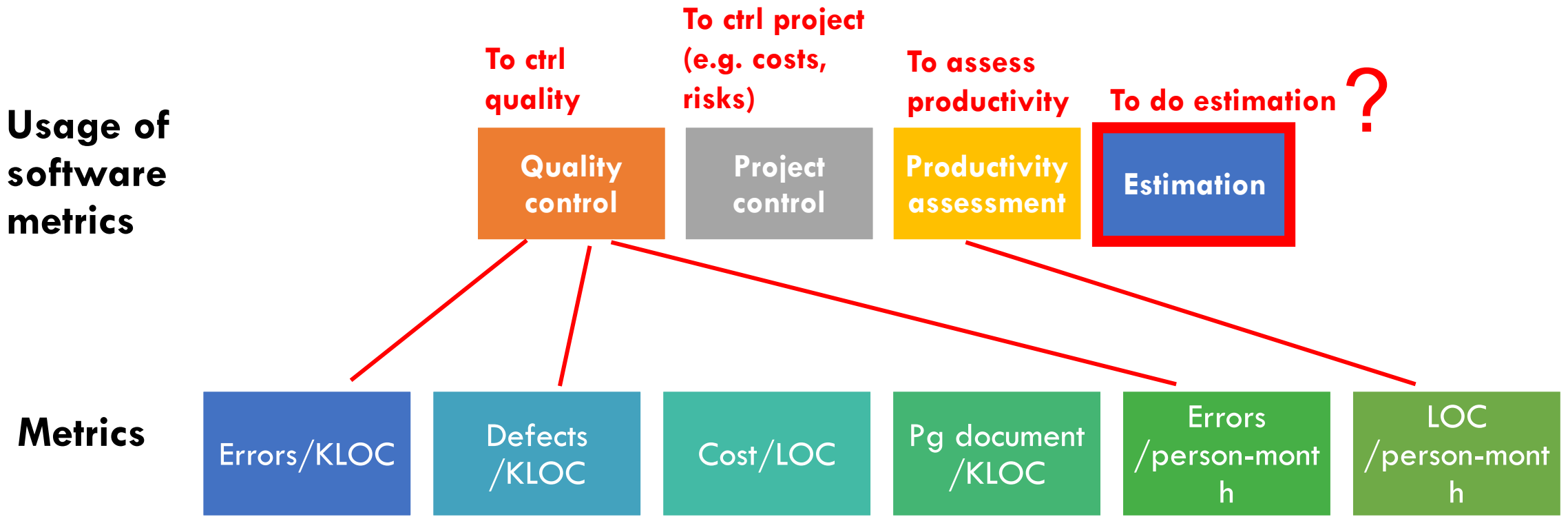
Based on Defects/KLOC, P(A) has **25%** more defects than P(B). *(0.0980-0.0781)/0.0781 * 100%*

Use of Software Metrics

4.2 Software Metrics

- **Quality control**
  - Measures of the fitness of use of the work products that are produced.
- **Project control**
- **Productivity assessment**
  - Measures of software development output as a function of effort and time applied.
- **Estimation** (using historical metrics)
  - What was software development productivity on past projects?
  - What was the quality of the software that was produced?
  - How can past productivity and quality data be extrapolated to the present?
  - How can it help us plan and estimate more accurately?

EXERCISE: Match the "Usage of software metrics" to the correct "Metrics"



Indicators

4.2 Software Metrics

	Non normalized		Normalized with KLOC	
	Project A	Project B	Project A	Project B
Total Line of Code (LOC)	306,000	256,000	-	-
Errors	288	233	288 / 306 = 0.9412	233 * / 256 = 0.9102
Defects	30	20	30 / 306 = 0.0980	20 / 256 = 0.0781
Page of	205	180		

E.g3. Which Project quality is higher?

- Project A has > **errors** than **Project B** [0.9412 vs 0.9102]
- Project A also has > **defects** than **Project B** [0.0980 vs 0.0781]

The "Normalized with KLOC" metrics give you an indicator that Project B quality is higher than Project A.

- **A metric or combination of metrics will provide you with insight into the software process, project or the product quality** so that improvement or adjustment can be made to the process or project.
- **E.g1**
  100 errors/KLOC is an indicator that the software is of poor quality.
- **E.g2. 0.01 defects/KLOC is an indicator** that the software is of _____ quality.

Indicators

4.2 Software Metrics

	Non normalized		Normalized with KLOC	
	Project A	Project B	Project A	Project B
Total Line of Code (LOC)	306,000	256,000	-	-
Errors	288	233	288 / 306 = 0.9412	233 * / 256 = 0.9102
Defects	30	20	30 / 306 = 0.0980	20 / 256 = 0.0781
Pages of Documentation	205	180	205/306 =	180/256 = 0.7031

- E.g. 100 errors/KLOC is an indicator that the software is of poor quality.
- A metric or combination of metrics that provides insight into the software process, project or the product itself so that improvement or adjustment can be made to

E.g. 4: Which Project has the higher degree of maintainability?

- **Project B** has > pages of documentations than Project A! (0.7031 vs 0.6699)
- This give you an indicator that *there are more info about projec B compared to P(A)*

***Conclusion**: A metric or combination of metrics will provide you an **INDICATOR** about the process, project or the product quality*

Types of Metrics

4.2 Software Metrics

3 Categories of Software metrics:

a. Product metrics

- Static metrics
- Dynamic metrics

b. Process metrics

- Time, resources, events
- Public
- Private

c. Project metrics

a. Product metrics

4.2 Software Metrics

- Concerned with the quality of the software itself
- 2 Classes of **product metric**

Static Metrics

- Have an indirect relationship with quality attributes. You need to try and derive a relationship between these metrics and properties such as complexity, understandability and maintainability.
- Measurements made of the system representations such as design, program or documentation
- Examples:
  - Line of code (LOC) – Measure size of a program. **more lines: more complex, more errors**
  - Length of variable names – **longer, more meaningful**
  - Depth of nested if-statement. **More depth means harder to understand and error-prone**
  - Cyclomatic complexity – Measure the complexity of a program by counting the number of independent paths through a program's source code from start to end. More path means more complex. Hence, testing & maintenance will be more difficult

If you measure these …

Dynamic Metrics (DM)

- Closely related to software quality attributes. Measurements of a program/system in execution
- **Examples** of measurement:
  - **time** required to start up an app *(**e.g.** time req. to start up the WA web)*
  - **execution time** required for a particular function. *(**e.g.** time required to generate a class code)*
  - the **number** of system failures. *(**e.g.** **2021**–12 failures, **2022**–14 failures)*

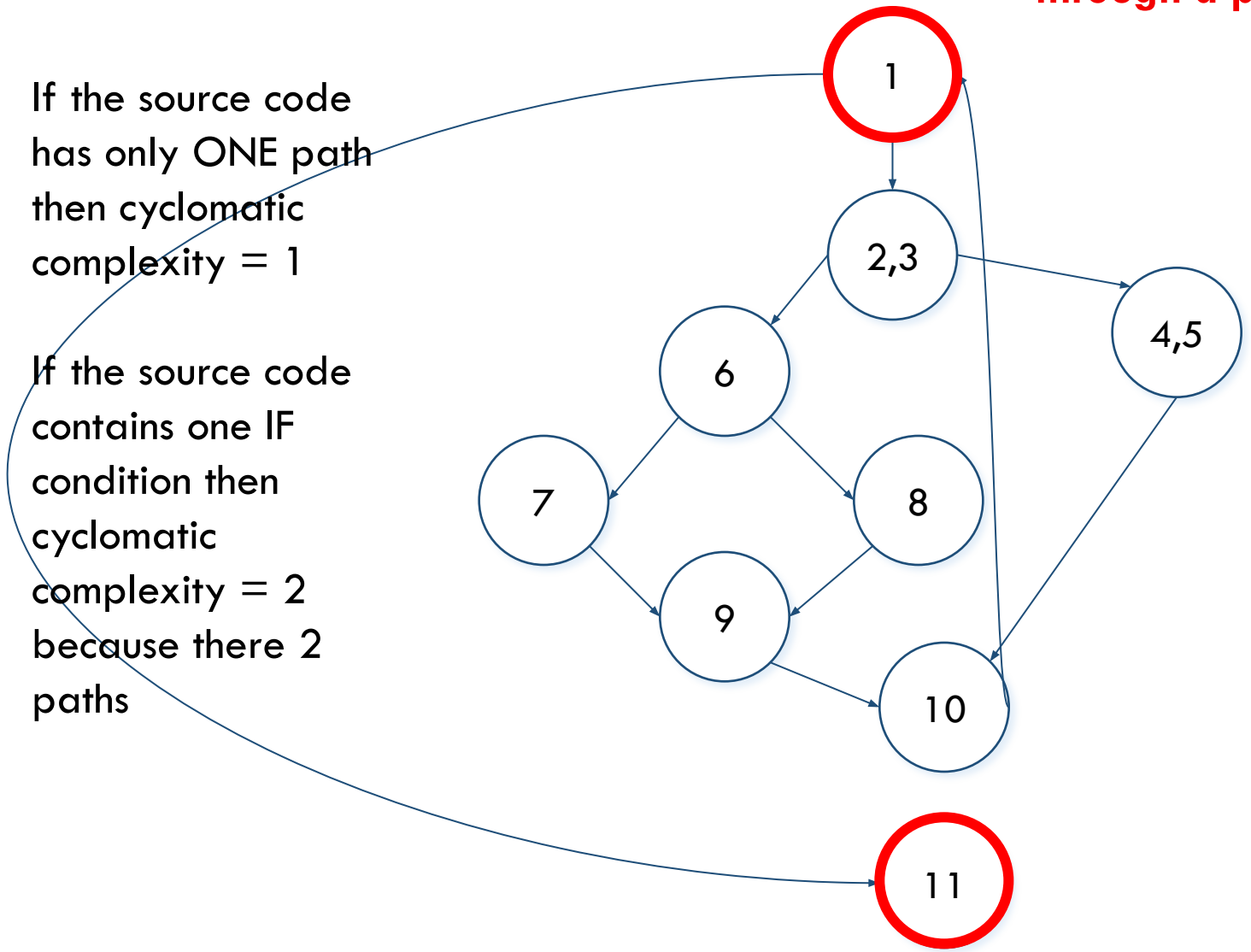
DM can be measured/collected during system testing or after system implementation

Cyclomatic Complexity

Measure the complexity of a program by counting the number of independent paths through a program's source code

If the source code has only ONE path then cyclomatic complexity = 1

If the source code contains one IF condition then cyclomatic complexity = 2 because there 2 paths



- Independent path: any path through the program that introduces at least one new set of processing statements or a new condition.
- E.g. this Flowchart shows there are 4 independent paths from point 1 to 11:
  - **Path 1: 1-11**
  - **Path 2: 1-2-3-4-5-10-1-11**
  - **Path 3: 1-2-3-6-8-9-10-1-11**
  - **Path 4: 1-2-3-6-7-9-10-1-11**

More path means _____

Types of Metrics

4.2 Software Metrics

Category of Software metrics:

a. **Product metrics**

- Static metrics **(LOC, length of identifiers, depth of nested if statement)**
- Dynamic metrics **(start up time, time required to execute a function, no of failures, etc**

b. **Process metrics**

- **Time, resources, events**
- **Public**
- **Private**

c. Project metrics

b. Process Metrics

Gives an indication of the system development processes

Types of process metric

- *The **time** **taken for a particular process to be completed***
  - This can be the total time devoted to the process, calendar time, the time spent on the process by particular engineers, and so on.
- *The **resources** **required for a particular process***
  - Resources might include total effort in person-days, travel costs or computer resources.
- ***The number of occurrences of a particular event***
  - Examples of events include:
    - **2** requirements changes received at the design stage
    - **5** run-time errors encountered during the first testing
    - **3** run-time errors encountered during the second testing

Event is "something" that happens

b. Process Metrics

4.2 Software Metrics

Examples of process metrics:

- Measures of errors uncovered before the release of the software
- **No of defects delivered** to and **reported** by users

(if this number is high, it indicates that testing processes are not effective)

- Work products/deliverables delivered
- Schedule conformance
- Human effort expended

(is the actual amount of labor that was used to finish a task)

(e.g. 20 SE were involved in the coding stage for 10 Days)
HUMAN EFFORT EXPENDED = 20 X 10 = 200 Person-Days

- Calendar time expended
- Time (hr or day)/SE task - can also be project metrics)
- Elapsed Time - time a request is received until evaluation is complete
- Effort (person-hr) to perform the evaluation
- Time elapsed from completion of evaluation to assignment of change order to personnel
- Effort required to make the change
- Time required to make the change
- Errors uncovered during work to make change

b. Process Metrics

4.2 Software Metrics

Public vs Private Process Metrics

- Public metrics
  - Integrated information that was originally private to individuals and teams
  - Examples:
    - Project level defect rates
    - Effort
    - Calendar times
    - Related data to uncover indicators to improve organizational process performance

- Private metrics
  - Process metrics that should be private to the individual software engineer and serve as an indicator for the individual only
  - Examples:
    - Defect rates (by individual)
    - Defect rates (by module)
    - Errors found during development

Types of Metrics

4.2 Software Metrics

Category of Software metrics:

a. Product metrics
   - Dynamic metrics
   - Static metrics

b. Process metrics
   - Time, resources, events
   - Public
   - Private

c. Project metrics

Examples

	Project A	P(B)	P(C)
Total LOC	300,000	300,000	300,000
Errors found per inspection	15	15	15
Defects reported by customer	20	25	30
Pages of documentation	200	200	200
Project duration *(Total mth taken to complete the proj)*	8	8	8
No of software engineers	10	10	10
No of changes to the project scope	12	15	20
Etc …			

c. Project Metrics

4.2 Software Metrics

- Use
to:
  -
  -
  -
  -
- Cor
metrics to:
  - Make necessary adjustments to avoid delays and mitigate potential problems and risks
  - Assess product quality on an on-going basis and modify the technical approach to improve quality

Project metrics

Examples

	Project A
Total LOC	300,000
Errors found per inspection	15
Defects reported by customer	20
Pages of documentation	200
Project duration *(Total mth needed/taken to complete the proj)*	8
No of software engineers	10
No of changes to the project scope	12
Etc …	

Project Metrics can be collected:

- **During estimation:**
  - Metrics collected from past projects are used as a basis from which effort and time estimates are made for current work
- During the execution of current project:
  - Compare scheduled dates & actual milestone dates to monitor progress
  - Make necessary adjustments to avoid delays and mitigate potential problems and risks
- During the progress of technical work:
  - Count errors uncovered per review
  - Distribution of effort per SE task
  - Pages of documentation per SE task
  - Delivered source lines per SE task
  - Function points per SE task

Types of Metrics

4.2 Software Metrics

3 Categories of Software metrics:

a. **Product metrics**
   - Dynamic metrics
   - Static metrics

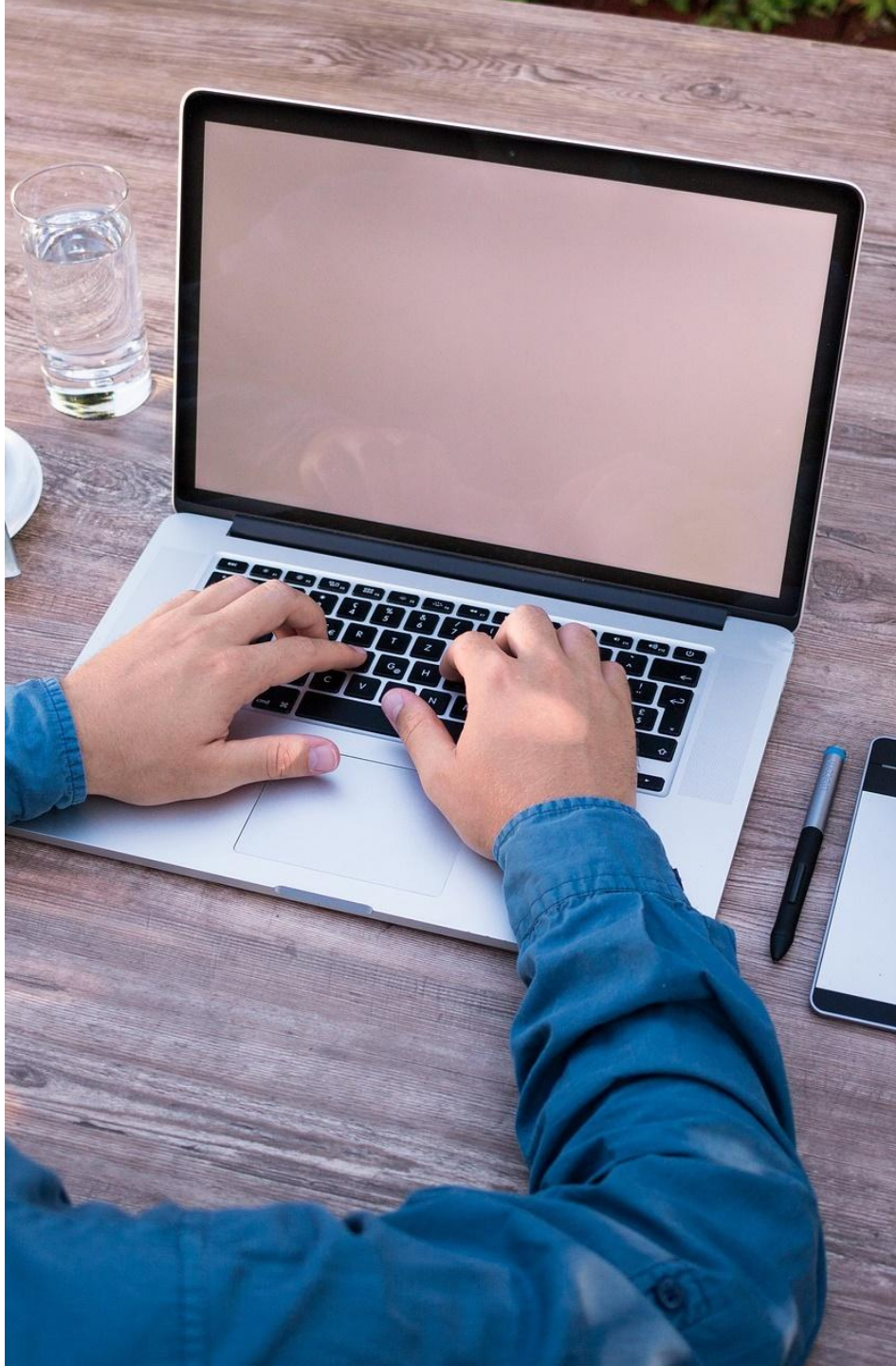
b. **Process metrics**
   - Time, resources, events
   - Public
   - Private

c. **Project metrics**

***Note**: Some of the process metrics are also metrics for project and/or product as well*

4.3 Software Measurement

- Direct and Indirect Measures
- **Size-Oriented Metrics**
- **Function-Oriented Metrics**



Direct and Indirect Measures

4.3 Software Measurement

Direct Measure

- more quantitative, easier to collect.
- E.g.:
  - Process & Project
    - Cost & effort applied
  - Product
    - LOC
    - Execution speed
    - Memory size
    - Defects over time

Indirect Measure

- measurement towards non-functional requirements but can still be expressed in quantitative form, quite difficult to measure.
- E.g.:
  - Functionality
  - Quality
  - Complexity
  - Efficiency
  - Reliability
  - Maintainability

Size-Oriented Metrics

4.3 Software Measurement

- Project A has 20 errors while Project B has 50 errors. Can we conclude that Project A is of better quality?
- Normalization of metrics between projects are required to reduce/get a value that is comparable in a fair manner between different projects.
- Two methods to obtain normalized metrics to compare different projects:
  - Size-oriented metrics
  - Function-oriented metrics

Size-Oriented Metrics

4.3 Software Measurement

Size-oriented metrics are derived by normalizing (dividing) with a related measure, e.g.:

- Quality measures over product size
- Productivity measures over the effort

Size-oriented Metrics - **Unnormalized**

Project	LOC	Duration (MTH)	$(000)	Pg. doc.	Errors	Defects	People (SE)
Alpha	12,100	24	168	365	134	29	3
Beta	27,200	62	440	1,224	321	86	5
Gamma	20,200	43	314	1,050	256	64	6
…	…	…	…	…	…		

Unnormalised metrics show, there is a **BIG** difference in quality between P(Alpha) & P(Beta)

Size-oriented Metrics - **Normalized**

Project	LOC/SE	LOC/MTH	$/LOC		Errors/KLOC	Defects/KLOC	People (SE)
Alpha	12,100/3 = 4,033	12,100/24 = 504	168,000/12100 = 13.88		1,340/121 = 11.07		
Beta	27,200/5 = 5,440	27,200/62 = 438	440,000/27200 = 16.18		3,210/272 = 11.80		

Normalised metrics show, there is only a **SMALL** difference in quality between P(Alpha) & P(Beta)

Size-oriented Metrics - **Unnormalized**

Project	LOC	Duration (MTH)	$(000)	Pg. doc.	Errors	Defects	People (SE)
Alpha	12,100	24				29	3
Beta	27,200	62				86	5
Gamma	20,200	43				64	6
…	…	…	…	…	…	…	…

Is Alpha team productivity **LOWER** than Beta team? __

Size-oriented Metrics - **Normalized**

Project	LOC/SE	LOC/MTH	$/LOC				
Alpha	12,100/3 = 4,033	12,100/24 = 504	168,000/12100 = 13.8				
Beta	27,200/5 = 5,440	27,200/62 = 438	440,000/27200 = 16.18		2 = 11.80		

Based on normalised metrics, is Alpha team productivity **LOWER** than Beta team? __

(assuming both projects are:
- using the same language
- the level of complexity is also the same, etc)

Size-oriented Metrics - **Unnormalized**

Project	LOC	Duration (MTH)	$(000)	Pg. doc.	Errors	Defects	People (SE)	Efforts
Alpha	12,100	24	168	365	134	29	3	72 person-mth
Beta	27,200	62	440	1,224	321	86	5	310 person-mth
Gamma	20,200	43	314	1,050	256	64	6	___ person-mth
…	…	…	…	…	…	…	…	…

Formula to calculate Efforts = *Duration * People*

Apha: 24*3 = 72 person-month

Beta : _____ = __ person-month

Gamma: ___ = __ person-month

72 person-month = 72 person working for 1 month

OR

1 person working for 72 months

OR

36 person working for 2 months

OR

Any other combinations which equal to 72

Size-oriented Metrics - **Unnormalized**

Project	LOC	Duration (Mth)	$(000)	Pg. doc.	Errors	Defects	People (SE)	Efforts
Alpha	12,100	24	168	365	134	29	3	72 person-mth
Beta	27,200	62	440	1,224	321	86	5	310 person-mth
Gamma	20,200	43	314	1,050	256	64	6	258 person-mth
…	…	…	…	…	…	…	…	…

To compare productivity

Size-oriented Metrics - **Normalized**

Project	LOC/SE	LOC/Mth		LOC/PERSON-MONTH
Alpha	12,100/3 = 4,033	12,100/24 = 504		12,100/72 = 168
Beta	27,200/5 = 5,440	27,200/62 = 438		27,200/310 = 88

To compare productivity

Use **LOC/Person-Mth** instead of LOC/Mth to make fair productivity comparison when project team size & duration are different

Normalized Size-oriented Metrics

4.3 Software Measurement

Examples

- Errors per KLOC (Errors/KLOC)
- Defects per KLOC (Defects/KLOC)
- $ per LOC ($/LOC)
- Pages of documentation per KLOC (Pages of doc/KLOC)
- Errors per person-month (Errors/person-month)
- LOC per person-month (LOC/person-month)
- $ per page of documentation ($/Page of doc)

KLOC = ??

Function-oriented Metrics

4.3 Software Measurement

- Measures the **functionality** delivered by the software system as a normalized value.
- This so-called "functionality" is derived indirectly using Function Point (FP) calculation.

To Measures the functionality delivered by the software system calculate Function Point (FP)

$$FP = Count\ Total \times [0.65 + 0.01\ \Sigma F_i]$$

is the sum of all FP entries

Fi are "complexity adjustment values" where i = 1 to 14

Function-oriented Metrics

4.3 Software Measurement

Function-oriented Metrics **(Measures the <u>functionality</u> delivered by the software system: <u>calculate</u>** Function Point (**FP**)

$$\mathbf{FP = Count\ Total \times [0.65 + 0.01\ \Sigma F_i]}$$

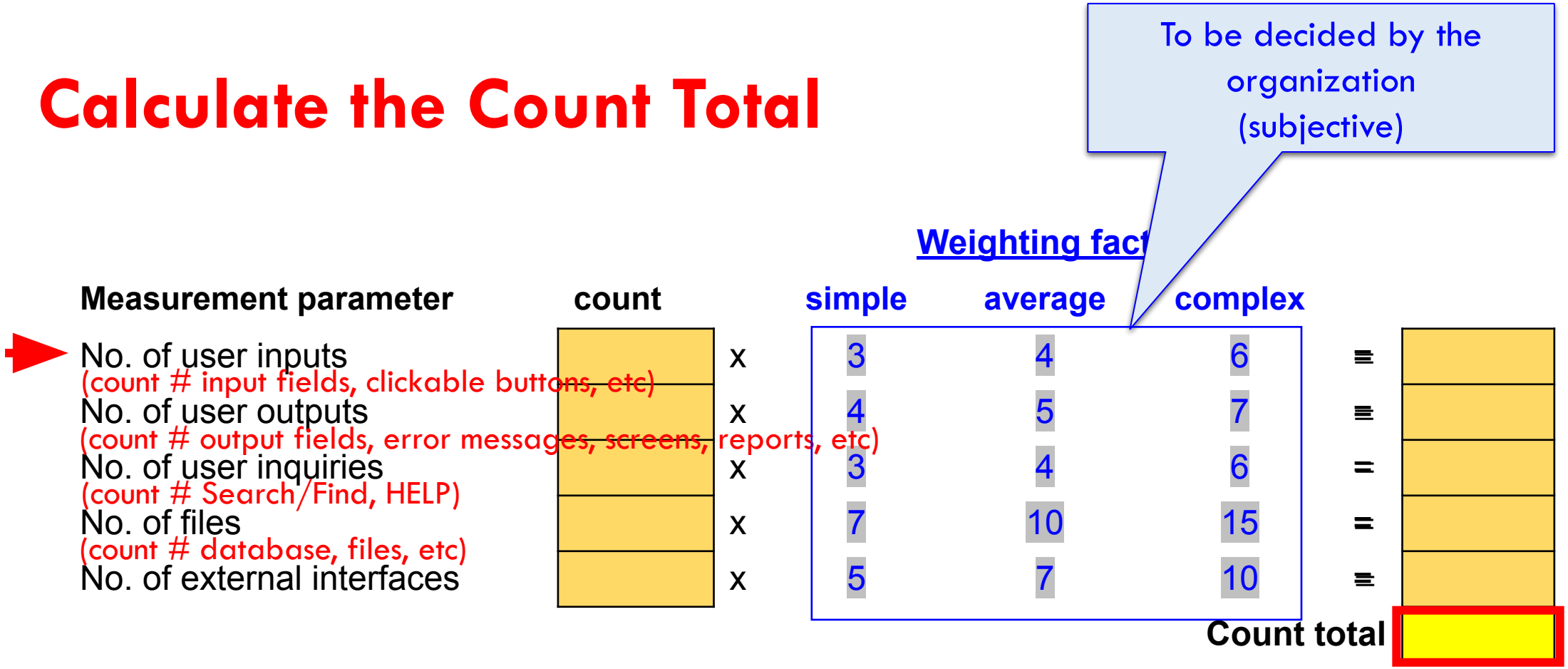
Count Total is the sum of the counts from the following *measurement parameters*:

- No. of user inputs – elements in the system which require an input from the user (e.g. input fields, clickable buttons, etc)
- No. of user outputs – elements in the system which produce an output (e.g. output fields, error messages, screens, reports, etc)
- No. of user inquiries – Search/Find, HELP)
- No. of files – database, files, etc
- No. of external interfaces - e.g. software communicates with a device (counted as 1 external interface)

Function-oriented Metrics

Calculate the Count Total

To be decided by the organization (subjective)



Function-oriented Metrics

FP = Count Total x [0.65 + 0.01 ΣF_i] (ΣF_i = total of **14 adjustment values based on 14 questions**)

Answer each question, using a scale from 0 to 5

1. Does the system require reliable backup and recovery?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require on-line data entry?
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

(For each Q, if the ans. is no, give a value 0, else decide whether it is 1,2,3,4 or 5)

(ΣF_i = 0 to 70)

**Responses are based on the following scale:*
0 – no influence
1 – incidental
2 – moderate
3 – average
4 – significant
5 – essential

Function-oriented Metrics

Exercise: Calculate Function Point

Measurement Parameter	Count	Weighting Factor	C * WF
Number of inputs	2	Complex (5)	10
Number of outputs	4	Simple (2)	8
Number of inquiries	7	Average (4)	28
Number of files	3	Average (7)	21
Number of external interfaces	2	Complex (8)	16
			Count Total= 83

Assuming $\Sigma F_i = 60$

FP = **Count Total** x $[0.65 + 0.01\ \Sigma F_i]$ = 83 x [0.65 + 0.01(60)]
= 83 x [1.25]
= **103.75**

Function-oriented Metrics

4.3 Software Measurement

Normalized Function-Oriented Metrics:

- Errors/ FP
- Defects/ FP
- $/ FP
- Pg doc/ FP
- FP/ person-month

Function-oriented Metrics

Exercise

a. Calculate the function point (FP) for **Project A and B.**

b. **120** pages of documentation is found in Project A, while **190** pages in Project B. Analyse **which project has higher degree of maintainability.** **(Answer this question by calculating the pages of doc per FP for each project)**

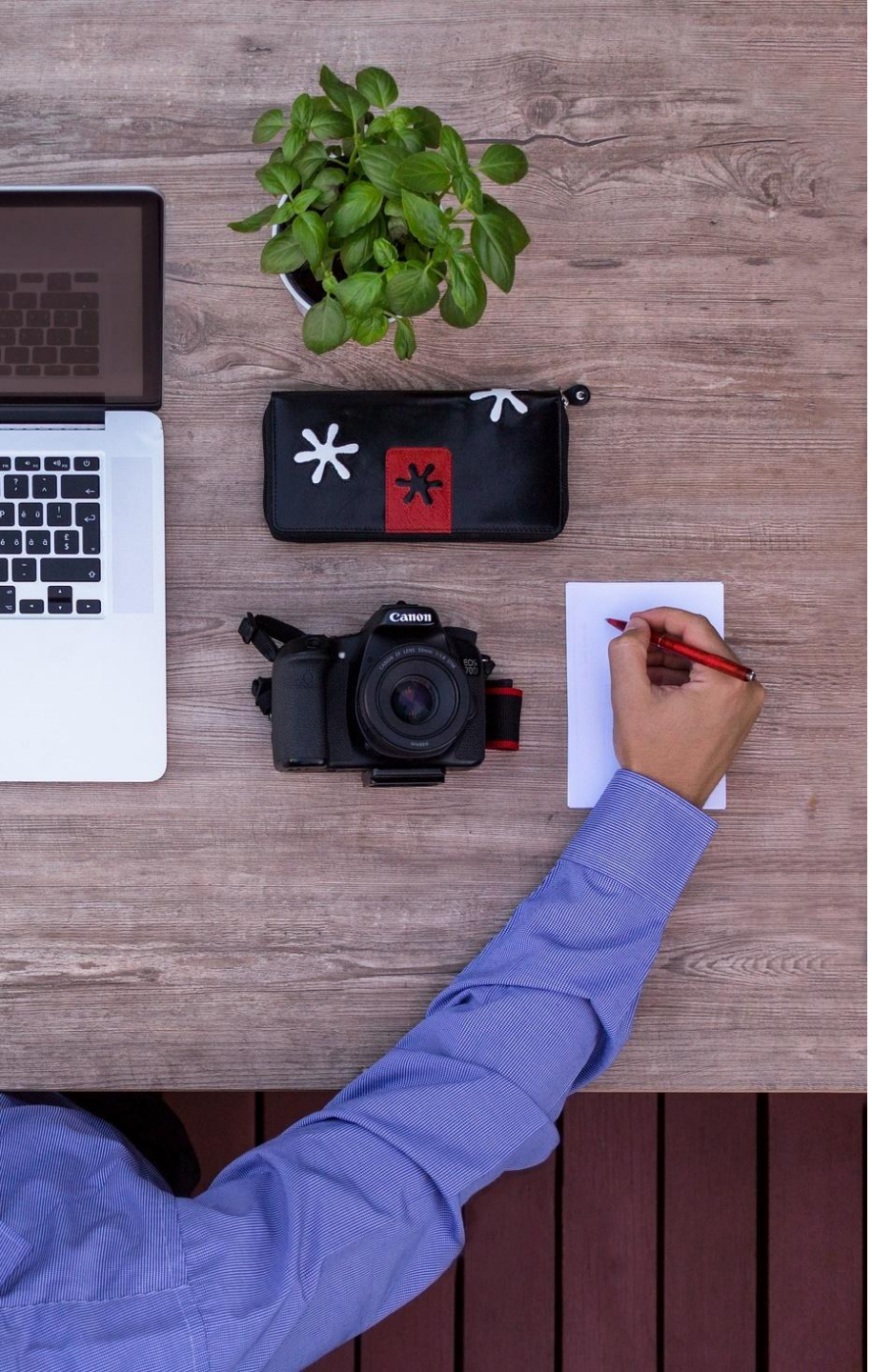
Measurement Parameter	Weighting factor	Project A's Count	Project B's Count
Number of inputs	Simple(2)	2	5
Number of outputs	Simple(2)	4	12
Number of inquiries	Average(4)	7	9
Number of files	Average(5)	3	7
Number of external interfaces	Complex(6)	2	3
Assuming $\Sigma F_i = 60$			

FP(A) = 67 x [0.65+(0.01 x 60)] = **83.75**

FP(B) = 123 x [0.65+(0.01 x 60)] = **153.75**

- Pages of doc per FP for **P(A)** = 120/83.75=**1.43**
- Pages of doc per FP for **P(B)** = 190/153.75 =**1.24**

Which project has higher degree of maintainability?
Project A because Project A has more Pages of Doc/FP than Project B (1.43 vs 1.24). This means that there are more info about P(A) than P(B)



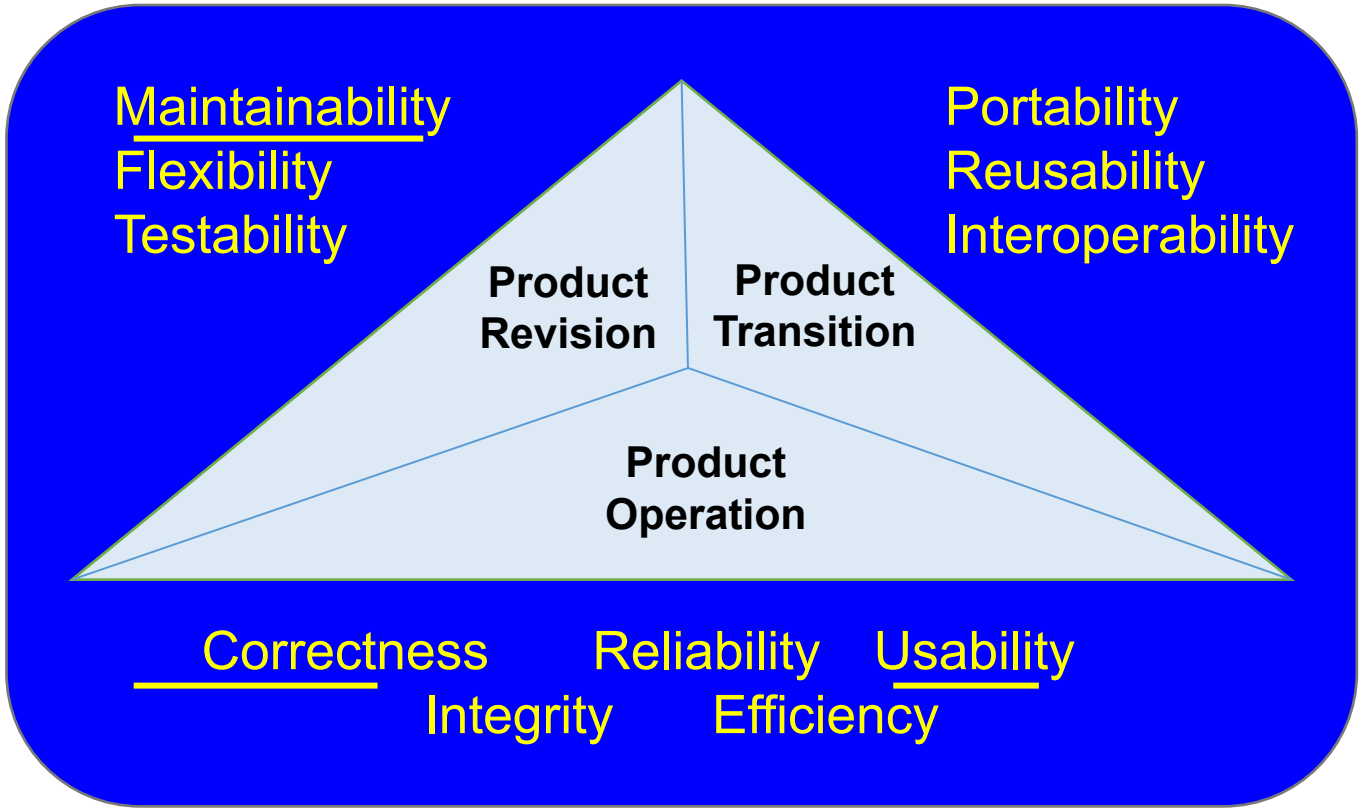
4.4 Metrics for Software Quality Attributes

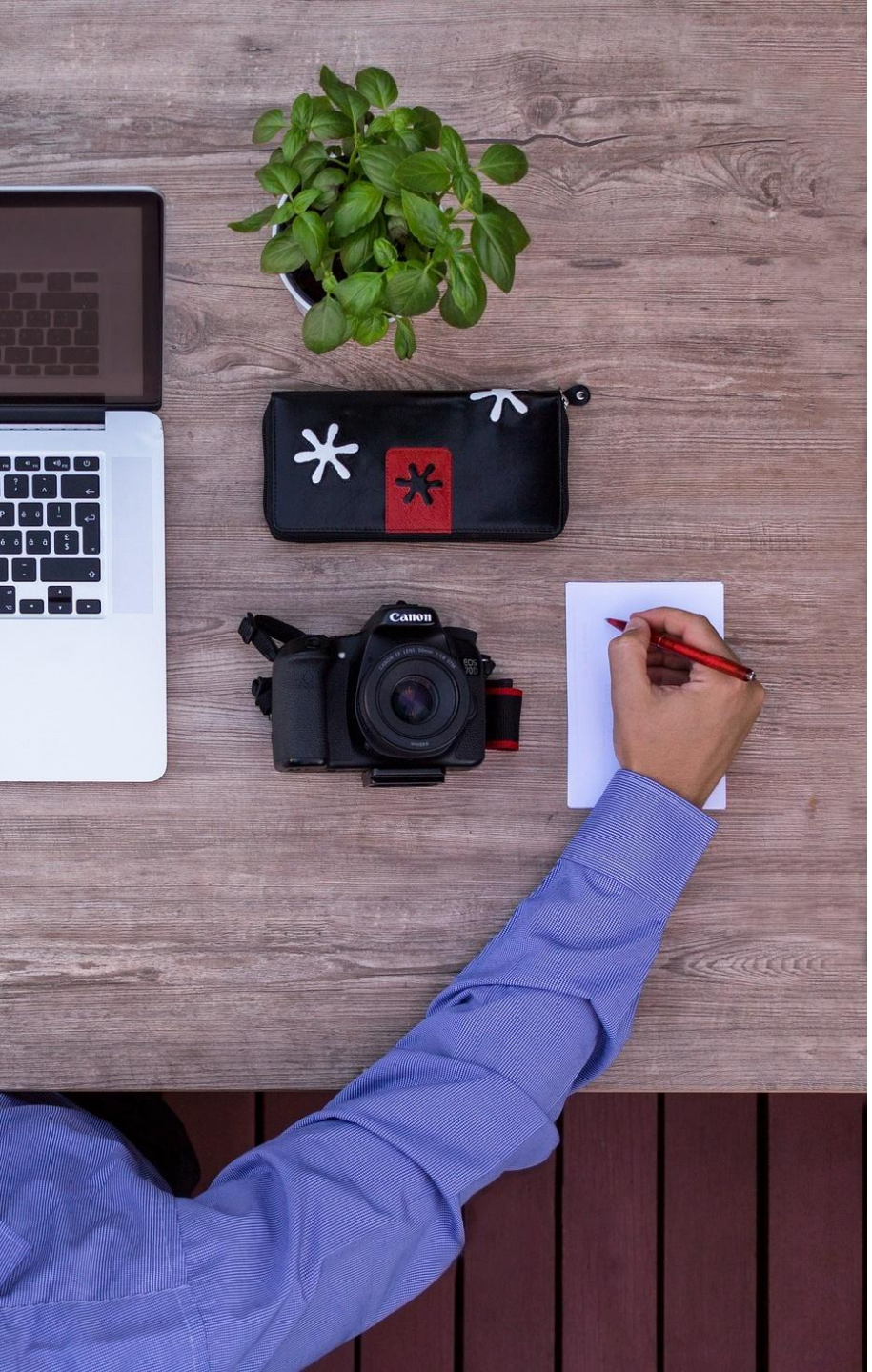
- McCall's Software Quality Factors
- Metrics for Measuring Software Quality
- Challenges in Measuring Software
- Characteristics of Good Software Metrics

McCall's Software Quality Factors

4.4 Metrics for Software Quality Attributes

- McCall's Software Quality Factors is a framework used to evaluate the quality of software products.
- 3 software quality factors categories:
  1) Product operation (while using it)
  2) Product revision (how easy it is to change it)
  3) Product transition (how easy it is to move the software to a different platforms or environments)





Metrics for Measuring Software Quality

4.4 Metrics for Software Quality Attributes

- What metrics can we use to measure the following software quality attributes?
  a. Usability
  b. Maintainability
  c. Correctness

Metrics for Measuring Software Quality

4.4 Metrics for Software Quality Attributes

a. Usability (Definition: **How easy it is to use the system**)

Can be **measured** by measuring:

1. The **time** required to learn how to perform a task for the first time using the system (e.g. cropping a video using a video editing software for the 1st time)
2. The **time** required to become moderately efficient in the use of a system (e.g. time needed to become moderately efficient in operating a car's infotainment system
3. The **user attitude** towards the system
   (using a questionnaire then count the no of –ve and +ve responses)

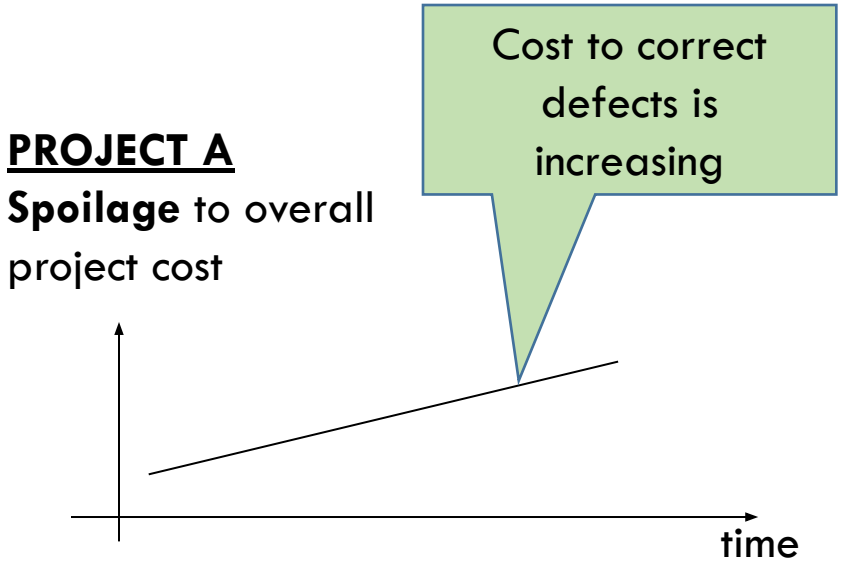
1. The **net increase in productivity**, measure the productivity when using the old process or system and measure again the productivity after a user has gained moderate efficiency when using the new process or system

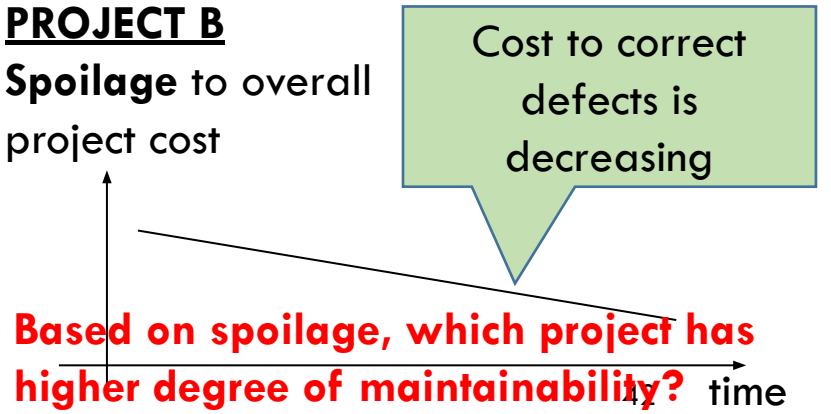
Metrics for Measuring Software Quality

4.4 Metrics for Software Quality Attributes

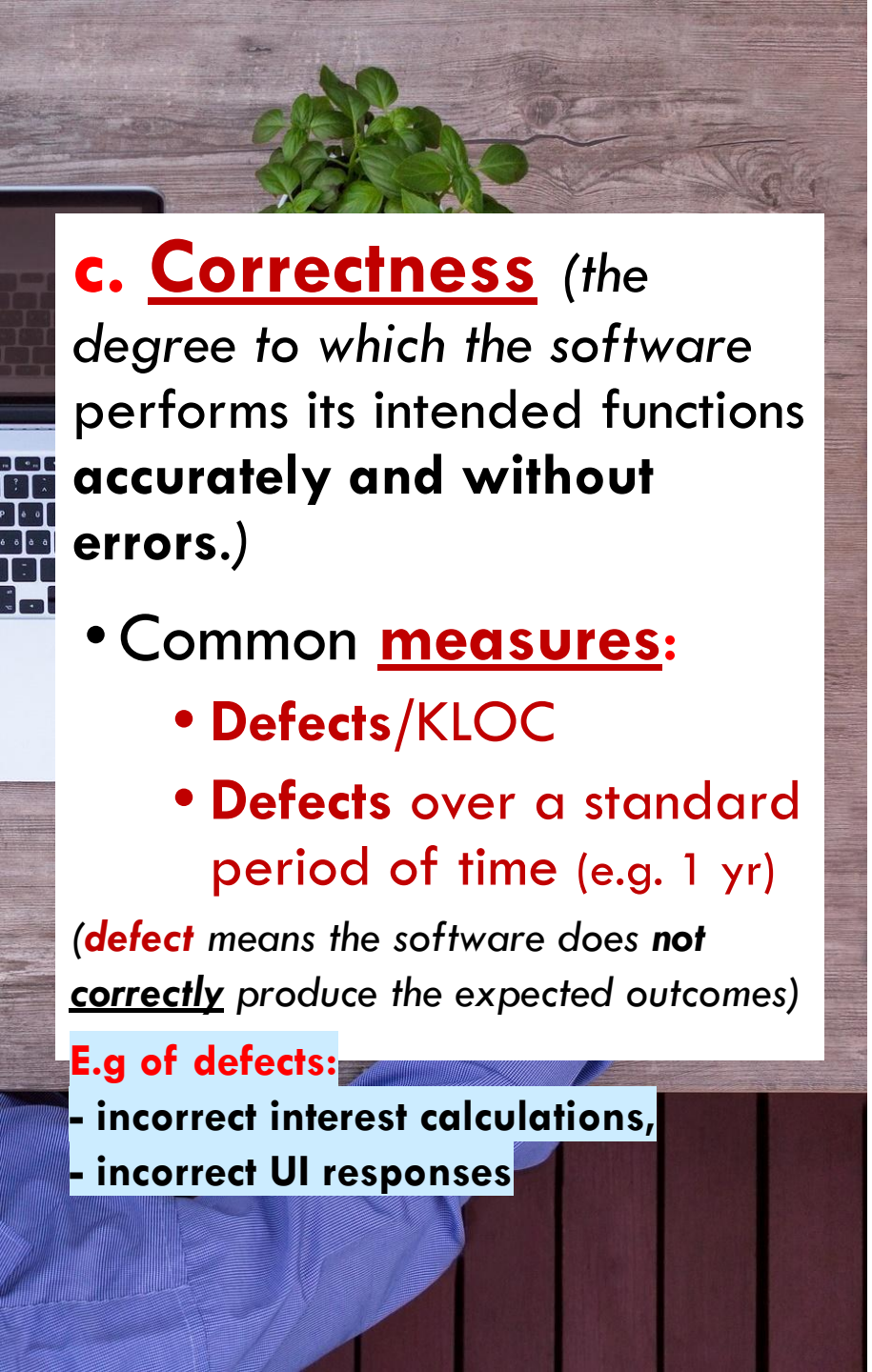
b. Maintainability

- The ease with which a program can be corrected if an error is encountered, adapted if its environment changes, or enhanced if the customer desires a change in requirements.
- No direct way to measure, so must use indirect **measures**:
  - **Mean-time-to-change (MTTC)**
    - **The time it takes to:**
      1. **analyse the change request,**
      2. **design an appropriate modification,**
      3. **implement the change,**
      4. **test it, and**
      5. **distribute the change to all users.**
    - **Highly maintainable programs will have a lower MTTC.**
  - **Spoilage**
    - **The cost to correct defects encountered after the software has been released to its end-users.**
    - When the ratio of spoilage to overall project cost is plotted as a function of time, a manager can determine whether the overall maintainability of software produced by a software development organization is improving.





Based on spoilage, which project has higher degree of maintainability?



c. Correctness *(the degree to which the software performs its intended functions* **accurately and without errors.**)

- Common **measures**:
  - **Defects**/KLOC
  - **Defects** over a standard period of time (e.g. 1 yr)

(***defect*** *means the software does* ***not correctly*** *produce the expected outcomes)*

E.g of defects:
- incorrect interest calculations,
- incorrect UI responses

Metrics for Measuring Software Quality

4.4 Metrics for Software Quality Attributes

Defects/KLOC calculation:

E.g. If a program has 100 defects and 10,000 LOC, the

Defects/KLOC = (100 / 10,000) * 1,000 = 10

Below are Defects/KLOC statistics

YEAR	2021	2022	2023
PROJECT	A	B	C
DEFECTS/KLOC	10	20	25

Data from the above table shows that the quality of completed projects is _____

Challenges in Measuring Software

4.4 Metrics for Software Quality Attributes

Normalised Size-oriented Metrics

- **LOC**
- **Errors**/KLOC
- **Defects**/KLOC
- **Pages of doc**/KLOC
- Errors/person-month
- LOC/person-month

Normalised Function-oriented metrics:

- Errors/ **FP**
- Defects/ FP
- $/ FP
- Pg doc/ FP
- FP/ person-month

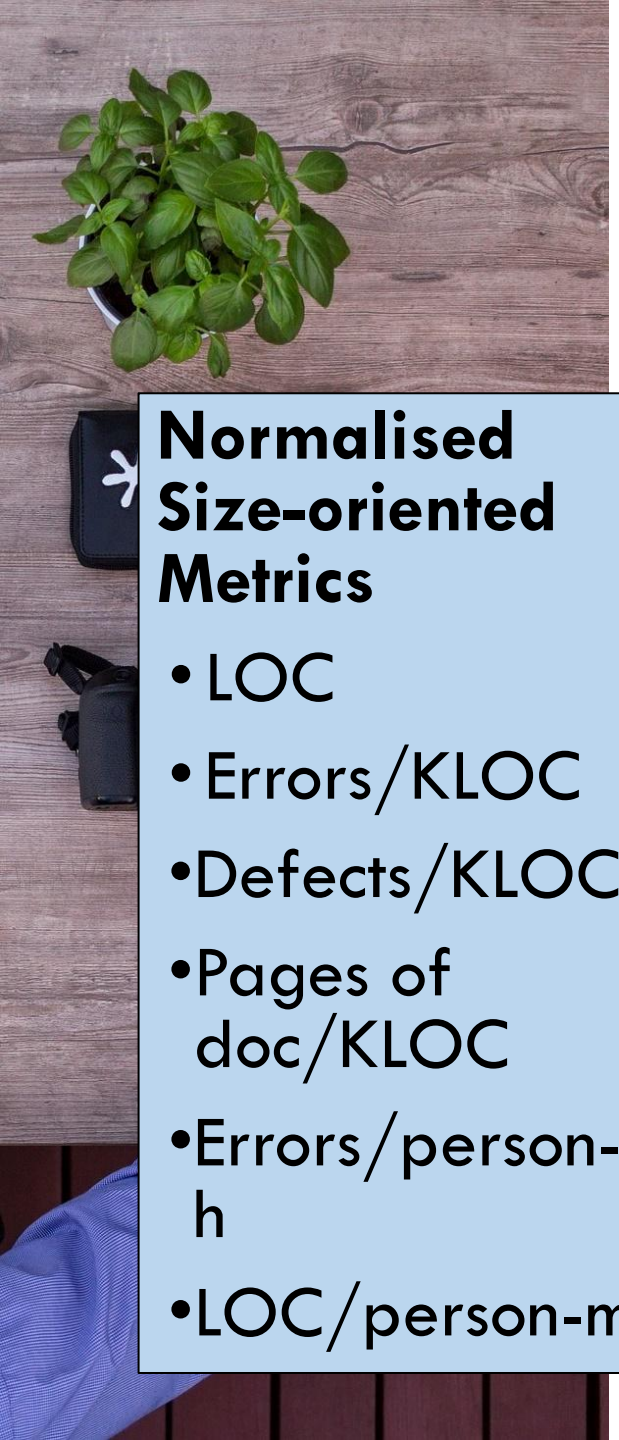
- Measurement is complex
- Too esoteric that very few professionals could understand
- Violate the basic intuitive notions of what high-quality software really is
- Many researchers attempted to develop a single metric that provides a comprehensive measure of software complexity
- Derived metrics might not be useful or suitable without realizing it. In other words, metrics might not prove anything
- Collecting measures is time consuming
- Difficult to determine what to measure and evaluating collected measures

Characteristics of Good Metrics

4.4 Metrics for Software Quality Attributes

- Consistent in its use of units and dimensions
  - The mathematical computation of the metric should use measures that do not lead to bizarre combinations of units.
  - E.g. use size and/or FP throughout all projects
- **Programming language independent**
  - Metrics should be based on the analysis model, the design model, or the structure of the program itself.
  - **LOC is programming language dependent**
    (Not suitable for comparing projects that uses different language)
    (FP is programming language independent?)
- **Simple and computable**
  - Relatively **easy to learn** how to derive the metric
  - Its computation **should NOT demand excessive amt of effort or time**
    - LOC is Simple and computable?
    - FP is Simple and computable?

FP = Count Total x [0.65 + 0.01 ΣF_i]
(ΣF_i = total of 14 adjustment values based on 14 Qs



Normalised Size-oriented Metrics

- LOC
- Errors/KLOC
- Defects/KLOC
- Pages of doc/KLOC
- Errors/person-month
- LOC/person-month

Characteristics of Good Metrics

4.4 Metrics for Software Quality Attributes

- Empirically and intuitively persuasive
  - Metrics should match the practitioner's notions about the product attribute under consideration
- Consistent and objective
  - Always yield results that are unambiguous
- An effective mechanism for high-quality feedback
  - **Should <u>motivate</u> team for software development improvement**
  - The metrics should lead to a higher-quality end product

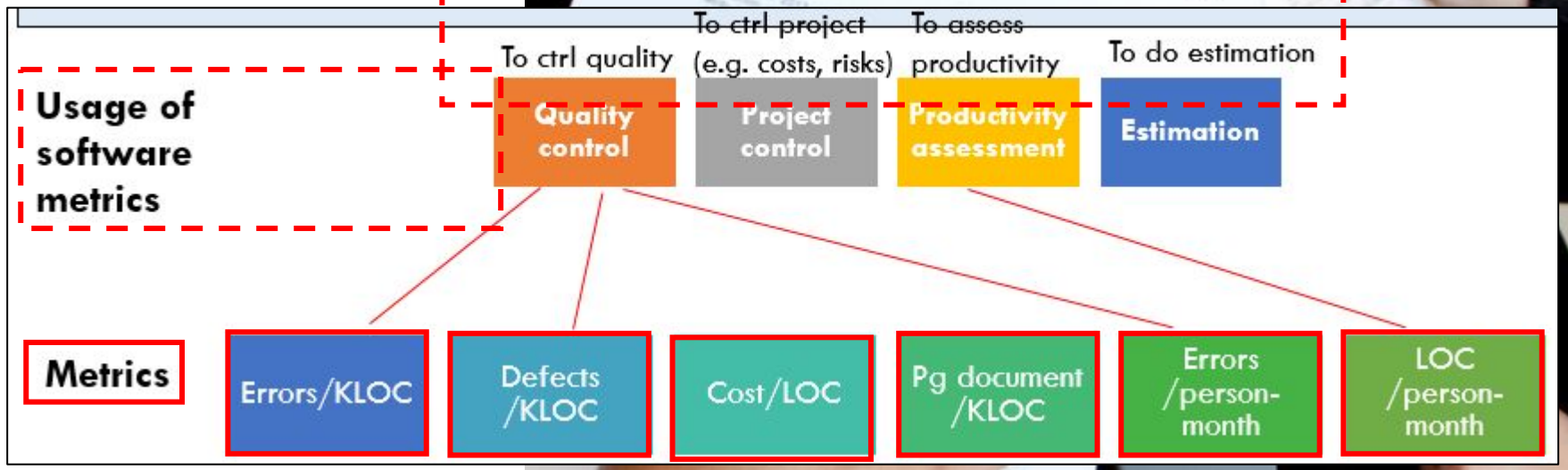
Summary

4.1 Introduction

4.2 Software Metrics

4.3 Software Measurement

4.4 Metrics for Software Quality Attributes



The end